# 가상환경의 필요성

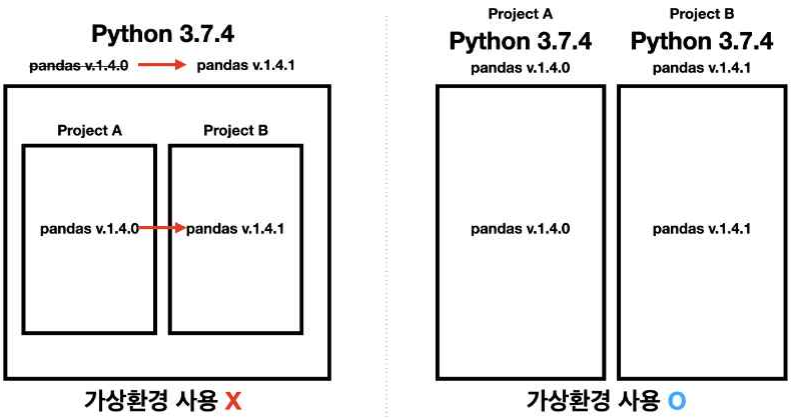
가상 환경은 프로젝트마다 격리된 환경(즉, 가상 환경)을 생성함으로써 프로젝트별로 패키지를 관리하기 위함이다.

## 파이썬의 패키지 관리 특징

파이썬은 같은 인터프리터 버전끼리 같은 패키지를 공유한다. 예를 들어, 가상 환경을 사용하지 않는 상황에서 A 프로젝트, B 프로젝트에서 Python 3.7.4를 사용하고 있다고 가정한다. 그럼 A 프로젝트에서 설치한 패키지를 B 프로젝트에서 따로 설치할 필요 없이 사용할 수 있다. 프로젝트마다 패키지를 매번 설치하지 않아도 되니 파이썬을 처음 입문한 경우 매우 편리하게 느껴진다.

하지만, 프로젝트마다 다른 버전의 패키지를 사용해야 한다면 치명적인 문제가 발생된다. 아래 그림과 함께 살펴보겠습니다. A 프로젝트에서 pandas 1.4.0 버전을 설치해 사용하고 있었다. 그럼 Python 3.7.4를 사용하는 모든 프로젝트에서 pandas 패키지는 1.4.0 버전으로 설치된다.

그런데 B 프로젝트에서 상위 버전을 사용하기 위해 pandas 1.4.1을 설치했다면 어떻게 될까? 이제 파이썬 3.7.4 버전의 모든 프로젝트는 pandas 패키지 버전이 1.4.1이 되고 이 상태에서 A 프로젝트를 진행하면 에러가 발생할 수 있다. 이러한 문제를 방지하기 위해 가상 환경을 사용하는 것이다.

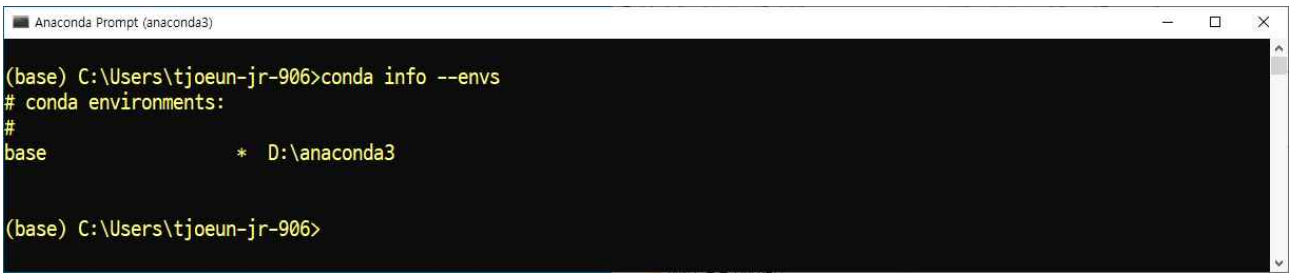



가상 환경을 사용하면 어떤 변화가 있을까? 위 그림 우측의 경우처럼 같은 인터프리터 버전을(Python 3.7.4) 사용하더라도 프로젝트별로 격리된 환경을 사용하기 때문에 패키지를 프로젝트별로 관리할 수 있다. 이처럼 가상 환경을 사용하면 프로젝트별로 안정적으로 패키지 버전을 관리할 수 있다.

# 아나콘다(Anaconda) 가상환경 생성, 삭제방법

## 가상환경 리스트 확인하기

conda info --envs



conda env list

```
명령 프롬프트
C:\Users\tjoeun-jr-906>conda env list
# conda environments:
#
base                  D:\anaconda3


C:\Users\tjoeun-jr-906>
```

## 가상환경 생성

conda create -n [가상환경 이름] python=[버전]

```
Anaconda Prompt (anaconda3)
(base) C:\Users\tjoeun-jr-906>conda create -n pythonConda37 python=3.7
```

```
Anaconda Prompt (anaconda3)
(base) C:\Users\tjoeun-jr-906>conda info --envs
# conda environments:
#
base                  *  D:\anaconda3
pythonConda37            D:\anaconda3\envs\pythonConda37


(base) C:\Users\tjoeun-jr-906>
```

## 가상환경 활성화하기

conda activate [가상환경 이름]

```
Anaconda Prompt (anaconda3)
(base) C:\Users\tjoeun-jr-906>conda activate pythonConda37

(pythonConda37) C:\Users\tjoeun-jr-906>
```

## 가상환경 내 패키지 설치 및 확인

패키지 설치하기

conda install [패키지 이름]

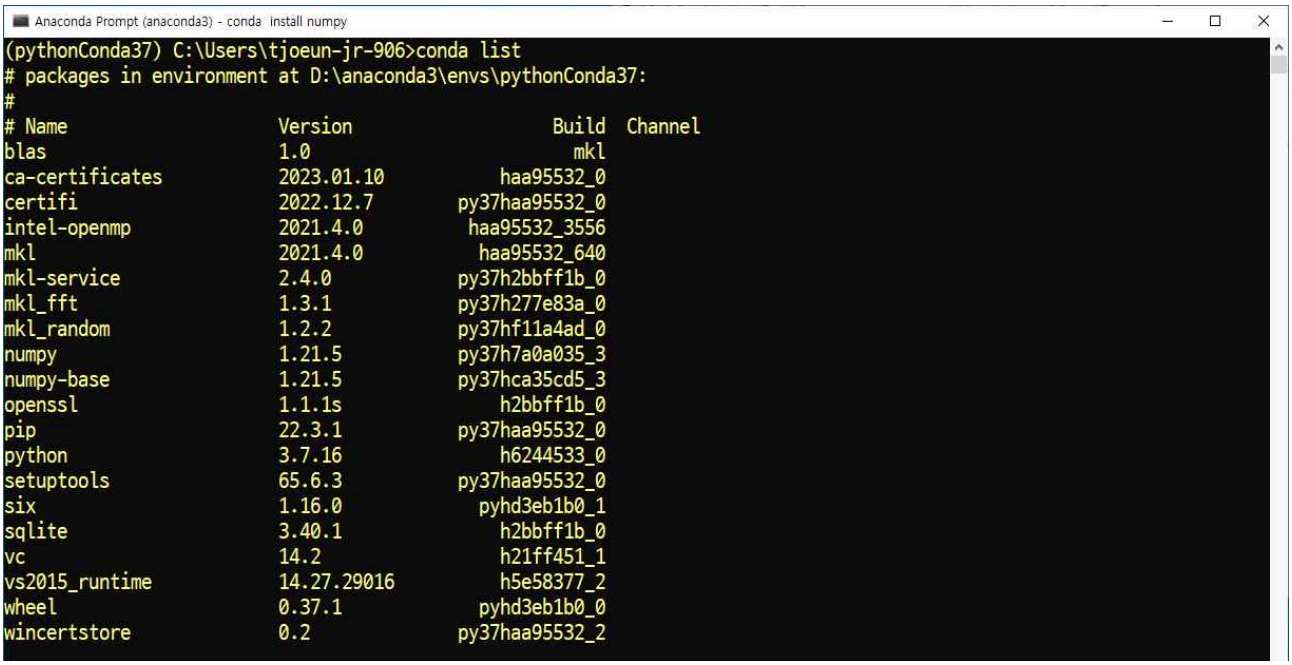
```
Anaconda Prompt (anaconda3) - conda  install numpy
(pythonConda37) C:\Users\tjoeun-jr-906>conda install numpy
```

pip는 의존성을 고려하지 않고 설치하고 conda는 라이브러리의 의존성을 고려하여 적절하게 설치한다.

```
conda install numpy
conda install pandas
conda install matplotlib
conda install tensorflow
conda install nb_conda => jupyter notebook
```

패키지 확인하기

```
conda list
```

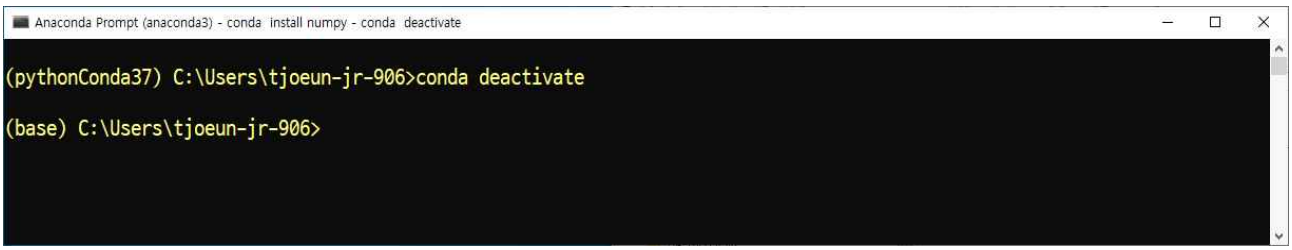



가상환경 활성화 없이 패키지 설치하기

```
conda install -n [가상환경 이름] [패키지 이름]
```

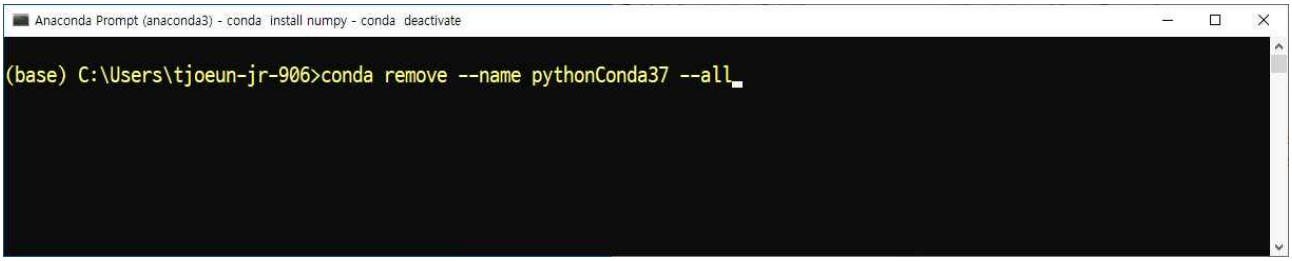
## 가상환경 비활성화하기

```
conda deactivate
```




## 가상환경 삭제하기

```
conda remove --name [가상환경 이름] --all
```



```
Anaconda Prompt (anaconda3) - conda install numpy - conda deactivate

(base) C:\Users\tjoeun-jr-906>conda info --envs
# conda environments:
#
base                  *  D:\anaconda3


(base) C:\Users\tjoeun-jr-906>
```

# jupyter notebook에 가상환경 kernel 추가 및 삭제

## 가상환경을 활성화 한 후 ipykernel 라이브러리를 설치한다.

```
명령 프롬프트
C:\Users\tjoeun-jr-906>conda activate py37

(py37) C:\Users\tjoeun-jr-906>conda install ipykernel
```

## jupyter notebook에 가상환경 kernel 설치하기

python -m ipykernel install --user --name 가상환경 이름 --display-name 커널 이름

```
명령 프롬프트 - conda install ipykernel

(py37) C:\Users\tjoeun-jr-906>python -m ipykernel install --user --name py37 --display-name py37
Installed kernelspec py37 in C:\Users\tjoeun-jr-906\AppData\Roaming\jupyter\kernels\py37

(py37) C:\Users\tjoeun-jr-906>
```

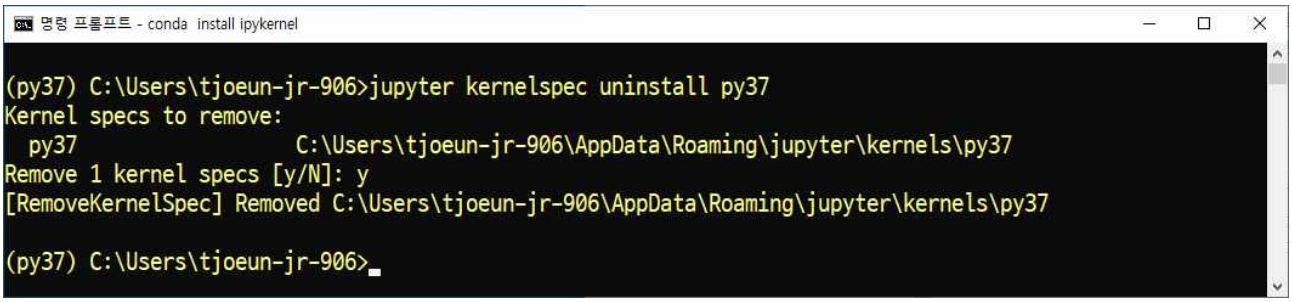
## 커널 목록 확인하기

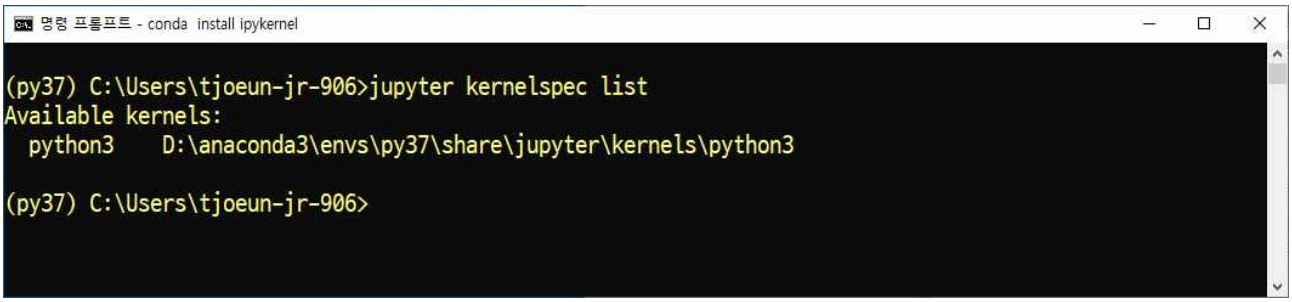
jupyter kernelspec list

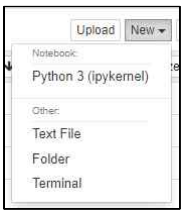
```
명령 프롬프트 - conda install ipykernel

(py37) C:\Users\tjoeun-jr-906>jupyter kernelspec list
Available kernels:
  py37        C:\Users\tjoeun-jr-906\AppData\Roaming\jupyter\kernels\py37
  python3     D:\anaconda3\envs\py37\share\jupyter\kernels\python3

(py37) C:\Users\tjoeun-jr-906>
```

## 커널 삭제하기

jupyter kernelspec uninstall 커널이름



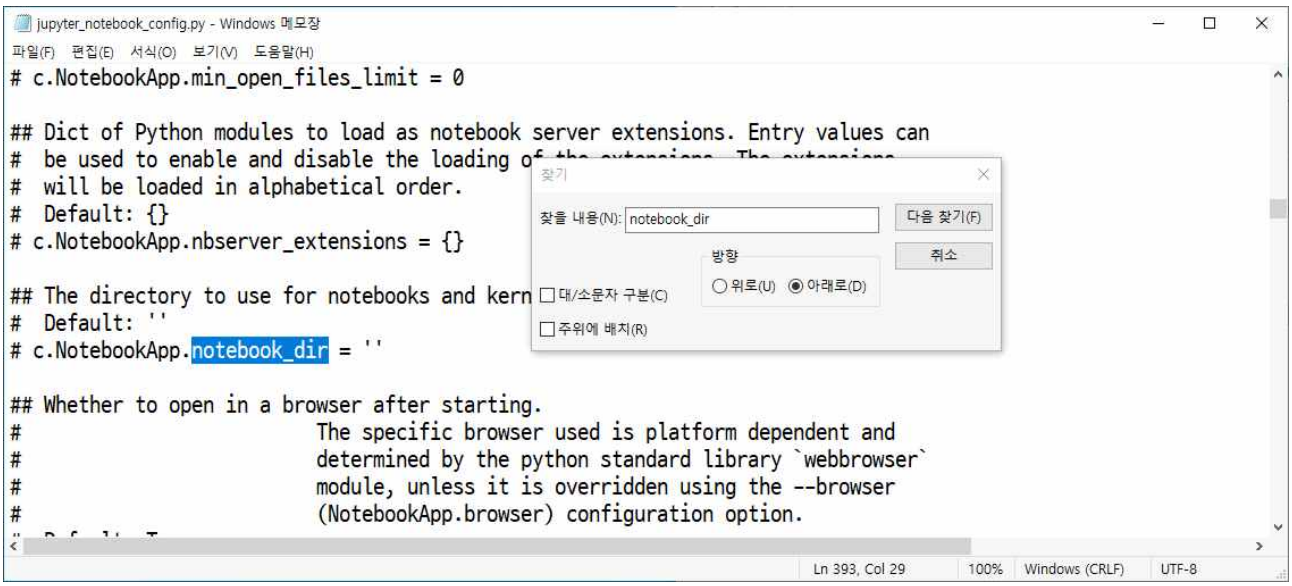



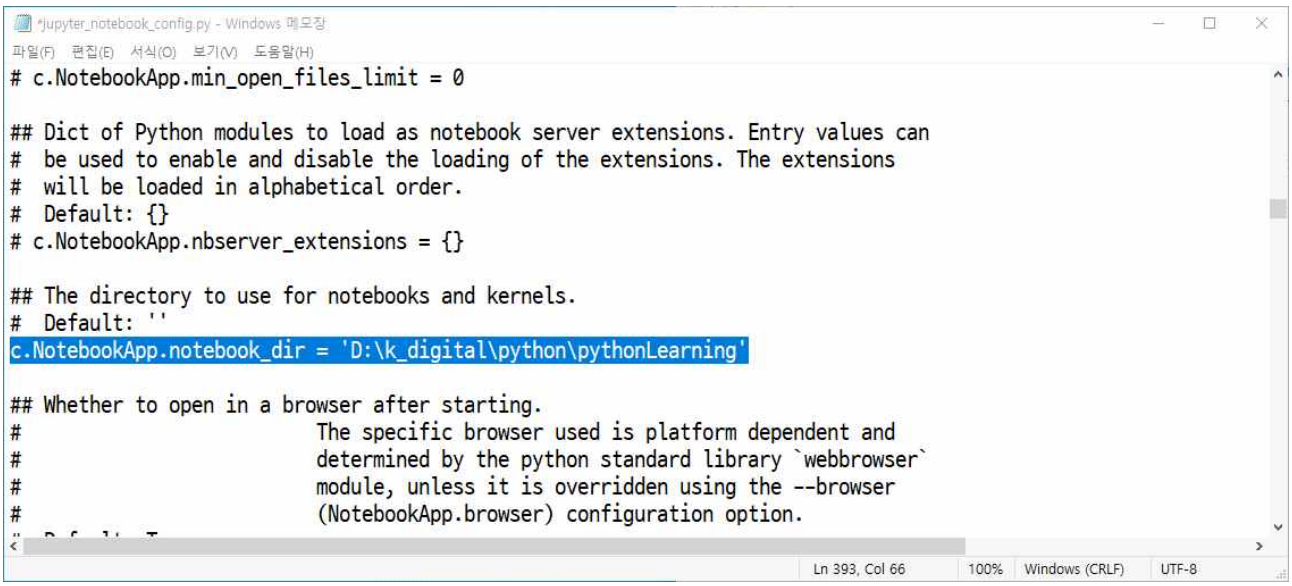
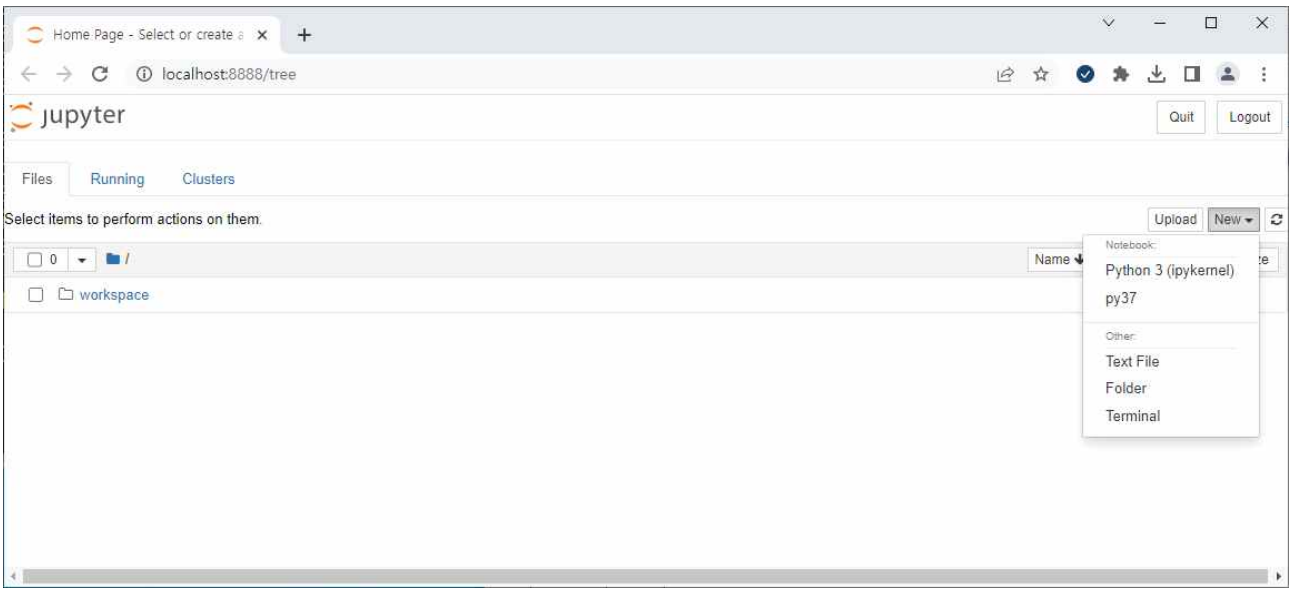

커널이 삭제가 안된다면 아래 경로에서 강제로 삭제하면 된다.

# jupyter notebook 홈 폴더 설정하기

jupyter notebook --generate-config를 실행한다.

C:\Users\<사용자 계정>\.jupyter 폴더의 jupyter_notebook_config.py 파일을 열어서 notebook_dir를 검색한다.




c.NotebookApp.notebook_dir = ''를 찾아 홈 폴더를 지정하고 #을 제거한다.